9.1 stringr: Strings

Lesson 1. Literal text: cut, paste, and change a string. The summary table is at the end.

Where you are. Lesson 1 of 5. Next is note 9.2.

This note is literal text. There is no pattern yet. The summary table is after the examples.

1. A string is text in quotes

`stringr` comes with `library(tidyverse)`. Every function starts with `str_`. The first argument is always the string (or a character vector), so the functions pipe.

```
library(tidyverse)

string1 <- "This is a string"
string2 <- 'A quote "inside" a string uses the other quote mark'
```

Special characters use a backslash. In the console you see the escape. `writeLines()` prints what the string actually contains.

```
x <- c("\"", "\\", "a\nb")
x
writeLines(x)
```

`\"` is a double quote. `\\` is one backslash. `\n` is a newline. `\t` is a tab.

---

2. Length and pieces

`str_length()` counts characters, including spaces. `NA` stays `NA`.

```
str_length(c("a", "R for data science", NA))
```

`str_c()` pastes strings together. `sep` goes **between** the arguments. `collapse` pastes the elements of one vector into a single string.

```
str_c("x", "y")
str_c("x", "y", sep = ", ")
str_c("prefix-", c("a", "b", "c"), "-suffix")
str_c(c("x", "y", "z"), collapse = ", ")
```

`sep` and `collapse` control different operations. `sep` is the separator placed between several arguments. `collapse` combines the elements of one vector into a single string.

If any argument is `NA`, `str_c()` returns `NA`. `paste()` instead converts a missing value to the characters `NA`. Convert the missing value to text first when it should remain in the string:

```
str_c("prefix-", c("a", NA, "c"), "-suffix")
str_c("prefix-", str_replace_na(c("a", NA, "c")), "-suffix")
```

---

3. `str_sub()` extracts and replaces

`str_sub(string, start, end)` takes a slice of characters. Counts are 1-based. Negative numbers count from the end. A too-long `end` just stops at the last character.

```
x <- c("Apple", "Banana", "Pear")
str_sub(x, 1, 3)
str_sub(x, -3, -1)
str_sub(x, 1, 1)
```

Assignment replaces that slice. The rest of the string stays.

```
str_sub(x, 1, 1) <- "Z"
x
```

---

4. Case

```
str_to_lower("THE WOODS")
str_to_upper("the woods")
str_to_title("the woods are lovely")
```

`str_to_title()` capitalizes each word. Locale matters for a few languages; the default is usually fine.

---

5. A first replacement

`str_replace()` changes the first match. `str_replace_all()` changes every match. A named vector is several replacements in one call. The names are the old pieces; the values are the new ones.

```
str_replace("back pack", "back", "foo")
str_replace_all(c("back", "lack", "black"), c("back" = "foo", "lack" = "foo"))
```

The second line also changes `"black"` to `"bfoo"`, because `"lack"` sits inside it. Patterns that mean “this word only” wait for note 9.2.

Summary

You want	Type
How many characters	`str_length(x)`
Join strings	`str_c(a, b, sep = ", ")`
Join a vector into one string	`str_c(x, collapse = ", ")`
Characters 1 through 5	`str_sub(x, 1, 5)`
Lower case, upper case, title case	`str_to_lower(x)`, `str_to_upper(x)`, `str_to_title(x)`
Change the first match	`str_replace(x, "old", "new")`
Change every match, or several literals at once	`str_replace_all(x, c("back" = "foo"))`

A newline inside a string is `"\n"`. A tab is `"\t"`. A quote inside a string is `"\""`. One backslash is `"\\"`.

---

6. Practice

1. From `full <- "To be or not to be, that is the question."`, use `str_sub()` to pull two pieces and `str_c()` to join them so `writeLines()` prints a two-line quote.
2. Replace `"back"` and `"lack"` with `"foo"` in one call. Try `c("back", "lack")` and then `c("back", "lack", "black")`.